rin

Twenty programs, read in order. Each one is short, runs, and teaches exactly one thing about how the machine actually works.

What rin is

> A small systems language that compiles to C, for people who want to know what the machine is doing.

The shape of it

rin is a language in the C family with the parts of C that hurt taken out and almost nothing added. It compiles to readable C, which your existing C compiler then turns into a binary — so it runs everywhere C runs, links against any C library, and needs no runtime of its own.

Everything is declared the same way, name first:

```
count:  i32   = 6;
Point:  struct = { x: f32; y: f32; }
main:   proc() -> i32 = { return 0; }
```

That order never changes. C puts the type first and wraps the name in declarator syntax nobody reads left to right; rin does not.

What it does differently

Sizes are in the names. `i32` is a signed 32-bit integer, on every machine, for ever. There is no `int` whose width depends on where you compiled it.

Only lossless conversions are silent. Widening an integer, or turning one into a float, happens on its own. Anything that can drop bits, a fraction or a sign needs a `cast`, at the place it happens. Literals have no suffixes: `2.5` is whatever its context wants, and it must fit.

Lengths travel with memory. `[N]T` is an array, `[]T` a slice that carries its length, `*T` the address of exactly one value, and `^T` C's raw pointer. All four read with `x[i]`. Indexes are checked wherever the length is known — at compile time for constants, in debug builds for the rest.

Order does not matter. A type or proc may be used before it is declared. No forward declarations, no header discipline, no include order to get right.

Memory is arenas. You ask once for a block, hand out pieces, and release the whole thing at once. Most memory in a real program shares a lifetime with some phase — a frame, a level load, a parse — and an arena says so out loud.

Generics and reflection, no macros. One definition instantiated per type, resolved while compiling, and a record describing every type that anything asks about. There is no macro system and no compile-time execution; that is the whole metaprogramming budget, deliberately.

No methods. Not now and not later. Data is data, procs are procs, and a proc pointer in a struct is the whole of what dispatch needs.

What it does not do

It does not manage memory for you. A slice into an arena that was cleared still points at it, arithmetic that overflows still wraps, and a raw `^T` is as unchecked as the C it came from. What the compiler refuses is the set of mistakes it can genuinely see: a conversion that loses information, a literal that does not fit, a constant index out of range, mismatched arities, two enums compared to each other, a `switch` that stopped covering its enum.

Is it real

The compiler is about 18,700 lines of C with a suite of several hundred discriminating checks. The largest program written in rin is a 3D engine of roughly 35,000 lines with a D3D11 renderer, which uses no C string functions at all.

Debug information points at your `.rin` files, not the generated C — you step through rin in a debugger, and the machine-code line tables name your source.

How to read these lessons

Twenty short programs, in order. Each one compiles, runs, and teaches one thing. Read the source first — the files carry their own commentary, including a "What to notice" and a "Try this" at the bottom — then read the page here for the framing.

The order follows the arc K&R uses: write a whole program, then types and sizes, then control flow, then procs, and only then pointers. You cannot see why an address is worth passing until you have watched a copy fail to come back.

Installing

> You need two things: the rin compiler, and a C compiler for it to hand its output to.

1. Get the compiler

The packaged toolchain is a directory containing four things:

```
rin-windows-x64/
    rin.exe          the compiler
    rinbind.exe      generates rin declarations from a C header
    libclang.dll     used by rinbind
    std/             the standard library, as rin source
```

`std` must sit beside `rin.exe`. The compiler resolves `import "std/..."` relative to its own location, and refuses to start if it cannot find it — a missing `std` is a broken install, and saying so once is better than one confusing error per import.

Put the directory wherever you like, then either add it to `PATH` or point `RIN_HOME` at it:

```
setx RIN_HOME C:\devel\rin-windows-x64
```

Check it:

```
rin --version
```

If that prints a version, the compiler is installed.

2. Get a C compiler

rin emits C, so something has to compile it. Any of these works:

- **clang-cl** — the default, and what the lessons are tested against
- **clang**
- **Microsoft C compiler** `cl.exe` — the whole lesson suite is tested against this too

If you have Visual Studio Build Tools or the Clang toolchain installed, you already have one.

3. Run a lesson

Two steps, because there are two compilers:

```
rin compile src/lessons/00_hello_world.rin -o build/hello.c --header build/hello.h
clang build/hello.c -I %RIN_HOME% -I %RIN_HOME%\std -o build/hello.exe
build\hello.exe
```

The two `-I` flags are how the C compiler finds `core.h` and `reflect.h`, which live beside the compiler along with `std`.

4. Or let the script do it

To build and run all twenty lessons at once:

```
powershell -ExecutionPolicy Bypass -File scripts/check_all.ps1
```

It finds the compiler through `RIN_HOME`, then `PATH`, then the default location, compiles every lesson, links it, runs it, and reports.

5. Or use `rin build`

For a project of your own, describe the build once in a `build.rin` and let the compiler drive cmake:

```
build_name:  ^const char = "hello";
build_entry: ^const char = "src/main.rin";
```

Then:

```
rin build
```

which transpiles, generates a `CMakeLists.txt` and runs cmake and ninja. This repository has one that builds all twenty lessons as separate programs.

Building the compiler from source

Only needed if you are working on rin itself. It wants cmake, ninja and clang-cl, then:

```
python bunyan.py build
```

which builds the compiler, copies `std` next to it, and packages the result.

If something goes wrong

`no input files` — `rin` with nothing to do says this, like any compiler. Give it a file.

`the compiler cannot find its own std` — `std/` is not beside `rin.exe`. Re-copy it, or pass `--no-std` if you genuinely mean to compile without it.

`a std directory next to the source shadows the compiler's own` — you have a folder called `std` next to the file you are compiling, and its contents differ from the shipped one. Rename it.

Missing `core.h` from the C compiler — the `-I` flags are wrong. They must point at the directory holding `rin.exe`, and at its `std` subdirectory.

00 — Hello, world

> The smallest complete program, and the declaration order everything else follows.

The idea

Every rin declaration reads the same way: a name, a colon, what it is, then `=` and its value. `main: proc() -> i32 = { ... }` is a name called `main`, which is a proc taking nothing and returning an `i32`, whose value is a block.

That order never changes. A variable is `count: i32 = 6`. A struct is `Point: struct = { ... }`. A proc is the same shape with a body. C puts the type first and wraps the name in declarator syntax that nobody reads left to right; rin puts the name first, always, and the rest follows.

What the lesson shows

- `main` returns `i32`, exactly like C — 0 means success
- `printfmt` comes from `std/print.rin`, which has to be imported
- `import` is not `#include`: it brings rin declarations in, not text

Why this is here

You will read this shape several thousand times. Getting used to it now costs one lesson; getting used to it later costs every lesson.

Before you move on

You can say what each part of `main: proc() -> i32 = {}` means, out loud.

Source: `src/lessons/00_hello_world.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/00_hello_world.rin — 8 lines

```
import "std/print.rin"

// A complete rin program looks like a small C program, but declarations use
// `name: Type = value` order.
main: proc() -> i32 = {
    printfmt("hello from rin\n");
```

```
    return 0;
}
```

01 — Values and expressions

> Arithmetic, and which conversions the compiler will do for you.

The idea

Expressions look like C. Conversions do not. Rin converts on its own only when nothing can go wrong: an integer to a wider one, and an integer to a float. Anything that can lose bits, drop a fraction or change sign needs a `cast`, written at the point it happens.

Literals have no suffixes. `2.5` becomes whatever the context wants — an `f32` when assigned to one, a `u8` when compared with one — and it must fit. Only where nothing says what a literal is does it default, to `i32` or `f64`, and there a tick names the type: `5000000000'i64`, `0.1'f32`.

What the lesson shows

- `u8` to `u32` to `i64` with no casts, because each step only widens
- `total / scale` dividing an `i32` by an `f32`, the int becoming a float
- `cast(wider, u8)` and `cast(ratio, i32)`: the two places a value can change
- typed literals where a call gives no context

Why this is here

Every later lesson that computes an offset, a size or a ratio meets these rules. The ones that remain are the conversions that are actually bugs: wrapping, truncating, flipping sign. They are worth meeting on a short program rather than in a renderer.

Before you move on

You can say which of `u8 -> u16`, `i32 -> u32`, `i32 -> f32` and `f32 -> i32` compile without a `cast`, and why.

Source: `src/lessons/01_values_and_expressions.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/01_values_and_expressions.rin — 45 lines

```
import "std/print.rin"

main: proc() -> i32 = {
    count: i32 = 6;
    scale: f32 = 2.5;           // the literal takes its type from the target
```

```
    total: i32 = count * 3 + 1;

    // Widening never loses anything, so it needs no ceremony.
    small: u8 = 200;
    wide: u32 = small;
    wider: i64 = wide;

    // int -> float is allowed too: it can lose precision, never sign or size.
    ratio: f32 = total / scale;

    // Going the other way can lose bits, so you write it down.
    back: u8 = cast(wider, u8);
    truncated: i32 = cast(ratio, i32);

    printfmt("count={} total={} ratio={}\n", count, total, ratio);
    printfmt("small={} wide={} wider={} back={} truncated={}\n", small, wide, wider, back, tru

    // With no target, a literal is i32 or f64. A tick names the type instead.
    printfmt("big={} single={}\n", 5000000000'i64, 0.1'f32);
    return 0;
}

// What to notice
//
// 1. Nothing here converts silently in a way that can change a number. u8 to
//    u32 to i64 only widens, and an int becomes a float -- that can round a huge
//    value, but never wrap it or flip its sign.
//
// 2. The two casts are the two places a value can actually change: 200 fits in
//    a u8 today, but `cast` is you saying so; 7.6 becomes 7.
//
// 3. `2.5` needs no suffix. It is an f32 because `scale` is one. A literal only
//    needs a tick when nothing around it says what it is, as in an argument to
//    printfmt.
//
// Try this
//
// - remove `cast(wider, u8)` and read the error: i64 to u8 can lose bits
// - write `c: u8 = 300;` -- a literal must fit its type
// - write `n: u64 = count;` -- i32 to u64 changes sign, so it is an error too
```

02 — Sizes

> Every value occupies a fixed number of bytes, and you choose how many.

The idea

In a language with one `int`, the size is chosen for you and you stop thinking about it. Here the size is written into the name. `i32` is a 32-bit signed integer: 4 bytes, on every machine, for ever. `u8` is one unsigned byte. There is nothing to look up and nothing that changes when you move platform — which C's `int` does not promise.

This is the first low-level decision there is, and rin makes you make it every time you declare something.

What the lesson shows

- `sizeof` on every scalar type, printed so you can read the numbers
- all pointers are the same size, because they all hold the same kind of thing
- `usize` is big enough to count any object, and matches the pointer size
- 200 stored in an `i8` reads back as -56, with no warning from anyone

Why this is here

That last point is the language's bargain stated early: the machine does exactly what you asked. The protection is not a check at runtime, it is that you wrote `i8` deliberately. Everything later in this series assumes you accept that trade.

Before you move on

You can predict `sizeof` for any scalar without running the program.

Source: `src/lessons/02_sizes.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/02_sizes.rin — 54 lines

```
import "std/print.rin"

// Every value your program holds occupies a fixed number of bytes, and you
// choose how many. That choice is the first low-level decision there is.
//
// In a language with one `int`, the size is picked for you and you stop
// thinking about it. Here the size is written into the name: i32 is a 32-bit
```

```
// signed integer, which is 4 bytes. Nothing is hidden.
//
// Run this and read the numbers before reading the explanation below it.

main: proc() -> i32 = {
    printfmt("i8   {} byte    u8   {} byte\n", sizeof(i8), sizeof(u8));
    printfmt("i16  {} bytes   u16  {} bytes\n", sizeof(i16), sizeof(u16));
    printfmt("i32  {} bytes   u32  {} bytes\n", sizeof(i32), sizeof(u32));
    printfmt("i64  {} bytes   u64  {} bytes\n", sizeof(i64), sizeof(u64));
    printfmt("f32  {} bytes   f64  {} bytes\n", sizeof(f32), sizeof(f64));
    printfmt("b32  {} bytes   c8   {} byte\n", sizeof(b32), sizeof(c8));

    // A pointer holds an address, and an address is the same size whatever it
    // points at. On this machine that is 8 bytes, because it is 64-bit.
    printfmt("*i32 {} bytes   *f64 {} bytes\n", sizeof(*i32), sizeof(*f64));

    // usize is "big enough to count any object in memory", which is why it is
    // the type of a size or a length. Notice it matches the pointer size.
    printfmt("usize {} bytes\n", sizeof(usize));

    // Size is not decoration. It decides the range of values that fit.
    // An i8 holds -128 to 127. Put 200 in one and the top bits are gone.
    small: i8 = cast(200, i8);
    printfmt("200 stored in an i8 reads back as {}\n", cast(small, i32));

    return 0;
}

// What to notice
//
// 1. The name tells you the size. You never have to look it up, and it does
//    not change when you move to another machine. `int` in C does.
//
// 2. Pointers are all one size, because they all hold the same thing: a
//    number that identifies a location. What lives there is a matter of type,
//    not of storage.
//
// 3. The i8 line is not a trick question. 200 does not fit in 8 signed bits,
//    so the bits that did not fit were discarded. The machine did not warn
//    you and neither did the compiler -- this language keeps C's bargain. The
//    protection is that you chose `i8` deliberately.
//
// Try this
//
// - change `small` to i16 and run again; the value survives, because 200 fits
```

```
// - print `sizeof(i32) * 8` to see the bit count you already knew
// - add `sizeof(*const c8)` and confirm const changes nothing about storage
```

03 — Control flow

> `if`, `for`, `switch`, and enums as the thing you switch on.

The idea

The shapes are C's, with one difference worth knowing: a `switch` over an enum that lists cases and writes no `default` must handle every member. Leave one out and the compiler names it.

Write `default:` and that check disappears entirely — so a large enum costs nothing. The rule only applies where you implied "I have covered them all" by omitting the default.

What the lesson shows

- `for` with an init, a condition and a step, and `continue` inside it
- an enum declared with three members and switched on
- `default:` catching the rest
- `&&`, `||` and `!`, which short-circuit exactly as in C

Why this is here

The exhaustiveness rule exists for one moment: the day you add a member to an enum and something silently stops handling it. The compiler will tell you which switches went stale.

Before you move on

You can say when the exhaustiveness check fires and how to opt out of it.

Source: `src/lessons/03_control_flow.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/03_control_flow.rin — 54 lines

```
import "std/print.rin"

Mode: enum = {
    Idle,
    Walk,
    Run,
}
```

```
main: proc() -> i32 = {
    sum: i32 = 0;

    for (i: i32 = 0; i < 5; i += 1) {
        if (i == 3) {
            continue;
        }
        sum += i;
    }

    mode: Mode = Mode.Run;
    switch (mode) {
        case Mode.Idle: {
            sum += 10;
        }
        case Mode.Walk: {
            sum += 20;
        }
        default: {
            sum += 30;
        }
    }

    // Logic is C's: && and || stop as soon as the answer is known, ! negates.
    moving: b32 = mode == Mode.Walk || mode == Mode.Run;
    if (moving && !(sum > 100)) {
        sum += 1;
    }

    printfmt("sum={}\n", sum);
    return 0;
}

// What to notice
//
// 1. The loop skips 3, so it adds 0 + 1 + 2 + 4. The switch takes `default`,
//    and the last `if` adds one more: 38.
//
// 2. `||` never evaluates its right side once the left is true, and `&&` never
//    does once the left is false. That is what makes `p != null && p* > 0`
//    safe to write.
//
// Try this
//
```

```
// - delete `default:` and read which members the compiler says are missing
// - add a member to Mode and see which switches it lands in
```

04 — Procs

> A piece of work with a name. You have been calling one since lesson 00.

The idea

The declaration is the same shape as everything else in the language:

```
name : proc (parameters) -> ReturnType = { body }
```

Read left to right: `double_it` is a proc, it takes an `i32` called `n`, it gives back an `i32`, and here is how. Nothing about that order changes for a struct, a variable or a type — which is the whole argument for it.

Two things matter more than the syntax.

Parameters are copies. A proc receives its own variable, initialised from whatever the caller passed. Writing to it changes nothing outside. That is the single fact the next lesson is built on: to change a caller's variable you have to pass its address, because passing its value cannot possibly work.

Order does not matter. A proc may be called by something declared above it. rin resolves names across the whole file, so there are no prototypes and no header discipline — C makes you write a forward declaration for this, and rin does not.

What the lesson shows

- one parameter, several parameters, and a `void` return
- a parameter written to inside the proc, with the caller's variable unchanged
- `factorial` calling itself, five frames alive at once before any returns
- a proc used before it is declared, with nothing to declare in advance
- a call used as an expression, nested inside another call

Why this is here

K&R spends a chapter on functions before it says a word about pointers, and that order is deliberate: you cannot understand why an address is worth passing until you have watched a copy fail to come back. Every lesson from 00 to 03 quietly used procs; this is where they stop being scenery.

Recursion earns its place for the same reason. Watching five `n`s exist at once is the first concrete picture of the stack, which lesson 10 will name.

Before you move on

You can say why `spend(budget)` left `budget` at 100, and what you would have to pass instead to change it.

Source: `src/lessons/04_procs.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/04_procs.rin — 105 lines

```
import "std/print.rin"

// A proc is a piece of work with a name. You have been calling one since the
// first lesson -- `main` is a proc -- and the lessons before this one quietly
// used a few more. This is the lesson that explains them.
//
// The declaration follows the same shape as everything else:
//
//   name : proc (parameters) -> ReturnType = { body }
//
// Read it left to right: `double_it` is a proc, it takes an i32 called `n`, it
// gives back an i32, and here is how.

double_it: proc(n: i32) -> i32 = {
    return n * 2;
}

// Parameters are copies. `n` above is this proc's own variable, initialised
// from whatever the caller passed. Writing to it changes nothing outside.
spend: proc(budget: i32) -> i32 = {
    budget = 0;      // only this copy
    return budget;
}

// Several parameters, separated by commas. Each needs its own type; there is
// no "and another one of those" shorthand.
clamp: proc(value: i32, low: i32, high: i32) -> i32 = {
    if (value < low) {
        return low;
    }
    if (value > high) {
        return high;
    }
    return value;
}

// A proc that returns nothing says so with `void`, and just ends.
announce: proc(label: ^const char, value: i32) -> void = {
```

```
    printfmt("{} = {}\n", label, value);
}

// A proc may call itself. Each call gets its own parameters and its own locals,
// stacked on top of the last, and they unwind as each one returns.
factorial: proc(n: i32) -> i32 = {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}

// Procs may be declared in any order. `helper` is used above by nothing, but it
// could be -- rin resolves names across the whole file, so there is no need to
// declare something before the thing that calls it. C makes you write a
// prototype for this; rin does not.
helper: proc() -> i32 = {
    return 42;
}

main: proc() -> i32 = {
    printfmt("double_it(21)     = {}\n", double_it(21));

    budget: i32 = 100;
    printfmt("spend(budget)     = {}\n", spend(budget));
    printfmt("budget afterwards = {}\n", budget);

    printfmt("clamp(15, 0, 10)  = {}\n", clamp(15, 0, 10));
    printfmt("clamp(-4, 0, 10)  = {}\n", clamp(-4, 0, 10));
    printfmt("clamp(7,  0, 10)  = {}\n", clamp(7, 0, 10));

    announce("helper()", helper());

    printfmt("factorial(5)      = {}\n", factorial(5));

    // A call is an expression, so it can go anywhere a value can.
    printfmt("nested            = {}\n", double_it(clamp(99, 0, 10)));

    return 0;
}

// What to notice
//
// 1. `budget` did not change. `spend` received a copy and set the copy to zero.
//    Every parameter in this lesson works that way, and it is the reason the
//    next lesson exists: to change a caller's variable you must pass its
```

```
//    address, not its value.
//
// 2. `factorial` calls itself five times before any of them return. Five sets
//    of `n` exist at once, each in its own frame on the stack. That storage is
//    handed back automatically as the calls unwind -- which is lesson 09.
//
// 3. `helper` is declared after everything that could call it and nothing
//    complains. Order does not matter at file scope.
//
// 4. `announce` returns `void` and its last statement is not a `return`. A proc
//    with nothing to give back simply reaches the end of its body.
//
// Try this
//
// - add `printfmt` inside `factorial` before the recursive call, and again
//   after it, to watch the calls stack up and then unwind
// - give `clamp` a fourth parameter and see the compiler reject every existing
//   call site; arity is checked, unlike C's older declaration style
// - write a proc that returns `f32` and call it from `main`
// - call `factorial(-1)` and reason about why it stops rather than recursing
//   forever
```

05 — Addresses

> A variable is a name for a place; a pointer holds that place's number.

The idea

Two operators do all the work, and they are exact opposites:

```
value&     the address of value
place*     the value at place
```

So `x&*` is `x` again. A pointer is an ordinary value — it has a size, it can be copied, it can be stored in a struct. It just happens to mean "the thing over there".

The lesson passes an address to a proc that then writes through it. The proc never sees the caller's variable; it sees a number, goes to that place, and writes. That is the entire mechanism, and every data structure later is built from it.

What the lesson shows

- two variables print two different addresses, close together but never equal
- nothing promises declaration order matches address order
- a proc changes its caller's variable by writing through an address
- repointing a pointer moves 8 bytes and no `i32` at all
- a `*i32` is the address of one `i32`: `p*` reads it, `p[1]` does not compile

Why this is here

Arrays, structs, arenas and every container in std are addresses plus arithmetic. If this lesson is fuzzy, the rest will feel like magic instead of like counting.

Before you move on

You can explain why `a&* = 5` and `a = 5` do the same thing.

Source: `src/lessons/05_addresses.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

`src/lessons/05_addresses.rin` — 79 lines

```
import "std/print.rin"

// A variable is a name for a place. The place has a number, and that number is
// what a pointer holds.
//
// Two operators do all the work:
//
//   value&    the address of `value`
//   place*     the value at `place`
//
// They are exact opposites. `x&*` is `x` again.

set_to_seven: proc(place: *i32) -> void = {
    // `place` is an address. Writing through it changes the caller's variable,
    // because there is only one place and both names refer to it.
    place* = 7;
}

main: proc() -> i32 = {
    a: i32 = 1;
    b: i32 = 2;

    printfmt("a = {} and lives at {}\n", a, cast(a&, uintptr));
    printfmt("b = {} and lives at {}\n", b, cast(b&, uintptr));

    // Two different variables, two different places. Close together, because
    // both are on the stack -- but never equal, and not necessarily in the
    // order you declared them. Nothing promises that `a` comes before `b`.
    if (cast(a&, uintptr) < cast(b&, uintptr)) {
        printfmt("a sits {} bytes below b\n",
                 cast(b&, uintptr) - cast(a&, uintptr));
    } else {
        printfmt("b sits {} bytes below a\n",
                 cast(a&, uintptr) - cast(b&, uintptr));
    }

    // A pointer is an ordinary value. It has a size, it can be copied, and it
    // can be stored -- it just happens to mean "the thing over there".
    p: *i32 = a&;
    printfmt("p holds {}, and p* is {}\n", cast(p, uintptr), p*);

    set_to_seven(p);
    printfmt("after set_to_seven, a = {}\n", a);

    // Point it somewhere else. Nothing about `a` or `b` changed; only what the
    // pointer refers to.
```

```
    p = b&;
    printfmt("now p* is {}\n", p*);

    return 0;
}

// What to notice
//
// 1. `a` was changed by a procedure that never saw `a`. It saw a number, went
//    to that place, and wrote there. That is the whole of what a pointer does.
//
// 2. The addresses are large and look arbitrary. They are -- they depend on
//    where the operating system put your stack today. Run it twice and they
//    will differ. The *distance* between them will not.
//
// 3. `p = b&` did not move any i32. It changed 8 bytes of pointer.
//
// 4. A `*i32` points at exactly one i32, so `p*` reads it and
//    `p[1]` does not compile. Pointing at many is a different type, and
//    lesson 06 is about those.
//
// Try this
//
// - print `cast(p&, uintptr)`: the pointer is itself a variable, so it has an
//   address of its own
// - call `set_to_seven(b&)` and confirm you can pass an address directly
//   without naming a pointer variable first
// - delete the `if` above and always subtract a's address from b's. If the
//   compiler put b lower, you will get something like 18446744073709551612
//   rather than a small number or a negative one. uintptr is unsigned, so
//   subtracting past zero wraps to the top of its range instead of going
//   negative. This is the same bargain as the i8 in lesson 01: the machine
//   does exactly what you asked and says nothing.
```

06 — Arrays, slices and pointers

> An array is a run of bytes, not a collection. A slice is that run plus its length.

The idea

There is no header, no length field, no bookkeeping. `[4]i32` is sixteen bytes with four values laid end to end, and its size is the element size times the count. That one fact explains why indexing is fast and why the compiler can compute an offset at compile time.

Rin gives that memory four shapes, all read with `x[i]` and addressed with `x[i]&`:

type	what it is	indexing
`[N]T`	N elements, stored inline	checked against N
`[]T`	a pointer and a length	checked against the length
`*T`	the address of exactly one `T`	`p*` reads it; `[0]` also works
`^T`	a raw address, C's pointer	unchecked

The slice is the useful one. `sum(xs: []const i32)` takes a whole array, part of one, or memory from anywhere else, and never needs a second "how many" argument. An array converts to a slice with the length taken from its type, and `values[1..3]` is a sub-slice: the same memory with a new start and length, no copy.

What the lesson shows

- `sizeof(values)` is `sizeof(i32) * 4`, with nothing added
- each element starts exactly one element-size after the last
- `values[0]&` is a `*i32`, and writing through it changes the array
- an array passes to a `[]const i32` parameter as is
- `values[1..3]` has two elements, and writing to it writes the array
- a raw `^i32` becomes a checked slice once you name its length: `raw[0..4]`

Why this is here

Every "pointer plus count" API in C is a slice waiting to happen, and every buffer overrun is that count going wrong. A constant index out of range does not compile. A variable index into an array or a slice is checked in debug builds, and the check compiles away in release. Only `^T` is unchecked, because only there is the length unknown.

Before you move on

You can compute the byte offset of `values[3]` without running anything, and say which of the four types a C function's `int *` should become.

Source: `src/lessons/06_arrays_and_pointers.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/06_arrays_and_pointers.rin — 103 lines

```
import "std/print.rin"

// An array is not a collection. It is a run of bytes, one element after
// another, with nothing in between. Rin gives that memory four shapes, all
// read with the same `x[i]` and addressed with the same `x[i]&`:
//
//     [N]T   an array: N elements, stored inline, length in the type
//     []T    a slice: a pointer and a length, travelling together
//     *T     a pointer to exactly one T
//     ^T   a raw pointer to "some" T -- C's pointer, unchecked

increment: proc(value: *i32) -> void = {
    value* += 1;   // a *T is read with *, never indexed past [0]
}

// A slice parameter takes any run of i32: a whole array, part of one, or
// memory from somewhere else. The length comes with it, so the loop needs no
// second argument and `xs[i]` is checked against it.
sum: proc(xs: []const i32) -> i32 = {
    total: i32 = 0;
    for (i: u64 = 0; i < xs.length; i += 1) {
        total += xs[i];
    }
    return total;
}

main: proc() -> i32 = {
    values: [4]i32 = {};
    for (i: i32 = 0; i < 4; i += 1) {
        values[i] = i * 10;
    }

    printfmt("{} {} {} {}\n", values[0], values[1], values[2], values[3]);

    // The whole array is one object, and its size is the element size times
    // the count. No header, no length field, no bookkeeping.
    printfmt("the array is {} bytes, an element is {}\n",
             sizeof(values), sizeof(i32));
```

```
    // Where the elements actually sit. Each one starts exactly sizeof(i32)
    // bytes after the last.
    base: uintptr = cast(values[0]&, uintptr);
    for (i: i32 = 0; i < 4; i += 1) {
        printfmt("  values[{}] at +{}\n", i, cast(values[i]&, uintptr) - base);
    }

    // So `values[i]` is not a lookup. It is arithmetic:
    //     address of element i  =  base + i * sizeof(element)

    // The address of one element is a *T: it points at that element and
    // nothing else.
    first: *i32 = values[0]&;
    increment(first);
    printfmt("after increment through a pointer: {}\n", values[0]);

    // An array converts to a slice; the length comes from its type.
    printfmt("sum of all four: {}\n", sum(values));

    // `a..b` takes a sub-slice: a included, b excluded. No copy -- it is the
    // same memory with a different start and length.
    middle: []i32 = values[1..3];
    printfmt("middle has {} elements: {} {}\n", middle.length, middle[0], middle[1]);
    printfmt("sum of the middle: {}\n", sum(middle));

    // A slice writes through to the array it came from.
    middle[0] = 99;
    printfmt("values[1] is now {}\n", values[1]);

    // A raw ^T is what C hands you: an address and no count. Indexing it is
    // unchecked, so you name the length once and get a checked slice back.
    raw: ^i32 = values;
    checked: []i32 = raw[0..4];
    printfmt("through the raw pointer, sum = {}\n", sum(checked));

    return 0;
}

// What to notice
//
// 1. The offsets are 0, 4, 8, 12. Nothing separates the elements. That is what
//    "contiguous" means, and it is why walking an array is the fastest thing a
//    processor does -- the next element is already in cache.
//
// 2. `sizeof(values)` is 16. The array is not a pointer. It is the bytes
```

```
//    themselves, sitting in this stack frame. A slice is two words, a pointer
//    and a length, wherever it points.
//
// 3. Indexes are checked where the length is known. A constant index out of
//    range -- `values[7]`, or `first[1]` on a *T -- does not compile. A
//    variable index into an array or a slice is checked at run time in debug
//    builds and stops the program with the file and line. Only ^T is
//    unchecked, because only there is the length unknown.
//
// 4. `sum` never asks how big its input is. The slice says.
//
// Try this
//
// - write `values[7]` and read the compile error; then make the index a
//   variable set to 7 and run it
// - pass `values[2..4]` to `sum`, then `values[3..3]` -- an empty slice is fine
// - change `[4]i32` to `[4]i64` and watch the offsets become 0, 8, 16, 24
// - open build/gen/06_arrays_and_pointers.c and find `RIN_INDEX`: that is the
//   check, and it compiles away in release builds
```

07 — Structs and enums

> Grouping values, and naming a small set of states.

The idea

A struct is several values with one name. An enum is a name for each of a small number of states, stored as an integer.

Both are what you would expect from C. The lesson is short on purpose — the interesting part is not the syntax, it is what the bytes look like, and that is the next lesson.

What the lesson shows

- a struct containing another struct, reached with `.`
- an enum member written `SpriteKind.Player`, always qualified
- zero-initialisation with `= {}`
- `printfmt` printing an enum by its member name, `Player`, rather than `0`
- a status enum marked `enum[must_use]`, and `_ = f();` to ignore one on purpose

Why this is here

Enum members are always written with their type in front. That is why two different enums can never be compared by accident, and why `Stat<>.count` in lesson 12 knows which enum you mean.

That is also why an enum works as an error code. A proc returns `LoadStatus`, and the caller compares it against `LoadStatus.Ok`. `must_use` closes the gap C leaves open: there, calling `load_level(2);` and ignoring the result compiles silently. Here it is an error. When ignoring it really is what you want, `_ = load_level(2);` says so in the code. Printing the status gives its name (`FileMissing`), so the message stays right when you add a member.

Before you move on

You can nest a struct in a struct and reach a leaf field. Uncomment `load_level(2);` and read the error.

Source: `src/lessons/07_structs_and_enums.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/07_structs_and_enums.rin — 49 lines

```
import "std/print.rin"
```

```
Point: struct = {
    x: f32;
    y: f32;
}

SpriteKind: enum = {
    Player,
    Pickup,
    Door,
}

Sprite: struct = {
    kind: SpriteKind;
    pos: Point;
}

// An enum that says whether something worked. `must_use` makes ignoring one a
// compile error, so a failure cannot be dropped by accident.
LoadStatus: enum[must_use] = {
    Ok,
    FileMissing,
    BadHeader,
}

load_level: proc(index: i32) -> LoadStatus = {
    if (index < 0) { return LoadStatus.FileMissing; }
    return LoadStatus.Ok;
}

main: proc() -> i32 = {
    s: Sprite = {};
    s.kind = SpriteKind.Player;
    s.pos.x = 3.0;
    s.pos.y = 4.0;

    // `{}` on an enum prints the member's name: this prints "kind=Player".
    printfmt("sprite kind={} pos=({},{})\n", s.kind, s.pos.x, s.pos.y);

    status: LoadStatus = load_level(-1);
    if (status != LoadStatus.Ok) {
        printfmt("load failed: {}\n", status);
    }

    // load_level(2);      // error: result of type 'LoadStatus' is ignored
    _ = load_level(2);     // ignored on purpose, and the reader can see that
```

```
    return 0;
}
```

08 — Layout and padding

> Two structs with the same three fields, different sizes.

The idea

A struct is a block of bytes with your fields at fixed offsets inside it. The compiler places them under rules you can predict: each field starts at an offset divisible by its own alignment, and the struct is padded at the end so an array of it stays aligned.

Reorder the declarations and the size changes. That is not a quirk — it follows from the placement rule, and once you know the rule you can do it deliberately.

What the lesson shows

- `Loose` and `Tight` hold the same values and are different sizes
- `offsetof` printed for each field, showing the gaps
- the padding at the end, and why an array of the struct needs it

Why this is here

This is where the series stops being about syntax. Everything from here — arenas, reflection, the C boundary — assumes you can look at a struct and see bytes rather than a bag of names.

Before you move on

You can reorder a struct to shrink it, and say by how much before compiling.

Source: `src/lessons/08_layout_and_padding.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/08_layout_and_padding.rin — 82 lines

```
import "std/print.rin"

// A struct is not a bag of fields. It is a block of bytes with your fields
// placed at fixed offsets inside it, and the placement follows rules you can
// predict once you know them.
//
// These two structs hold exactly the same three values. Only the order of the
// declarations differs.
```

```
Loose: struct = {
    flag:  u8;      // 1 byte
    value: i32;     // 4 bytes
    small: u8;      // 1 byte
}

Tight: struct = {
    value: i32;     // 4 bytes
    flag:  u8;      // 1 byte
    small: u8;      // 1 byte
}

// The same fields again, with the compiler told to pack them.
Packed: struct[packed] = {
    flag:  u8;
    value: i32;
    small: u8;
}

main: proc() -> i32 = {
    printfmt("Loose  is {} bytes, aligned to {}\n", sizeof(Loose), alignof(Loose));
    printfmt("Tight  is {} bytes, aligned to {}\n", sizeof(Tight), alignof(Tight));
    printfmt("Packed is {} bytes, aligned to {}\n", sizeof(Packed), alignof(Packed));
    printfmt("the three fields themselves are {} bytes\n",
             sizeof(u8) + sizeof(i32) + sizeof(u8));

    // Where each field actually sits. Subtracting the struct's own address
    // from a field's address gives you that field's offset.
    l: Loose = {};
    t: Tight = {};
    base_l: uintptr = cast(l&, uintptr);
    base_t: uintptr = cast(t&, uintptr);

    printfmt("Loose: flag at +{}, value at +{}, small at +{}\n",
             cast(l.flag&, uintptr) - base_l,
             cast(l.value&, uintptr) - base_l,
             cast(l.small&, uintptr) - base_l);
    printfmt("Tight: value at +{}, flag at +{}, small at +{}\n",
             cast(t.value&, uintptr) - base_t,
             cast(t.flag&, uintptr) - base_t,
             cast(t.small&, uintptr) - base_t);
    return 0;
}

// What to notice
```

```
//
// 1. Loose and Tight hold the same data and are not the same size. Nothing was
//    added to Loose except emptiness.
//
// 2. Look at where `value` sits in Loose: offset 4, not offset 1. An i32 wants
//    to start at an address divisible by 4 -- that is what "aligned to 4"
//    means -- so the compiler left three unused bytes after `flag` to get
//    there. Those three bytes are padding. You paid for them and cannot use
//    them.
//
// 3. Tight has no hole in the middle, because putting the 4-byte field first
//    means the two 1-byte fields land where they already fit. There is still
//    padding at the *end*: the struct's own size has to be a multiple of its
//    alignment, so that an array of them keeps every element aligned.
//
// 4. Packed has no padding at all -- and its alignment dropped to 1. That is
//    the trade. Reading `value` now costs more on most machines, and on some
//    it is not even allowed. Use `packed` when a layout is dictated by
//    something outside your program: a file format, a network packet, a
//    hardware register. Not to save memory.
//
// Try this
//
// - add a fourth field `other: u8;` to Tight and watch the size stay the same,
//   because it fits in padding that was already being paid for
// - make `value` an i64 in Loose and watch the padding grow to 8-byte steps
// - print `sizeof(Loose) * 1000` against `sizeof(Tight) * 1000` to see what
//   field order costs in an array you might actually allocate
```

09 — Procs and callbacks

> A proc has an address too, so it can be stored and passed.

The idea

`Op: alias = *proc(a: i32, b: i32) -> i32` names a type: pointer to a proc taking two `i32`s and returning one. A struct can hold one. A variable can hold one. Calling through it looks like calling anything else.

This is how you get behaviour that varies at runtime without any of the machinery a language with methods would need. rin has no methods and never will; a proc pointer in a struct is the whole answer.

What the lesson shows

- an alias naming a proc-pointer type
- a struct with a callable field
- swapping which proc a struct calls, at runtime

Why this is here

Every callback, every dispatch table, every "do this later" in a real rin program is this. It is also why `alias` exists — writing the pointer type out at each use would be unreadable.

Before you move on

You can add a third operation and swap it in without touching the struct.

Source: `src/lessons/09_procs_and_callbacks.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/09_procs_and_callbacks.rin — 27 lines

```
import "std/print.rin"

Op: alias = *proc(a: i32, b: i32) -> i32;

Adder: struct = {
    call: Op;
}
```

```
add: proc(a: i32, b: i32) -> i32 = {
    return a + b;
}

mul: proc(a: i32, b: i32) -> i32 = {
    return a * b;
}

main: proc() -> i32 = {
    table: [2]Op = {};
    table[0] = add;
    table[1] = mul;

    adder: Adder = {};
    adder.call = add;

    printfmt("{} {} {}\n", table[0](2, 3), table[1](2, 3), adder.call(10, 5));
    return 0;
}
```

10 — Storage and lifetime

> Where a value lives decides how long it lasts. There are three answers.

The idea

Static: one copy, created before `main` runs, alive until the program exits. `static` on it means "not visible outside this file", exactly as in C.

Local: created when the block is entered, gone when it exits. Fast, and the default for a reason.

Dynamic: memory you asked for explicitly, alive until you say otherwise.

You choose one every time you declare something, usually without noticing. This lesson is about telling them apart by looking at the declaration.

What the lesson shows

- a static counter that survives across calls
- a local that does not, and is fresh each time
- the file-scope/global distinction that `static` controls
- returning the address of a local, and why that is the classic mistake

Why this is here

The next lesson replaces "dynamic" with an arena, which is rin's actual answer. You need the three-way distinction first, because an arena is a way of managing the third one, not a fourth thing.

Before you move on

For any declaration you can say which of the three it is and why.

Source: `src/lessons/10_storage_and_lifetime.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/10_storage_and_lifetime.rin — 90 lines

```
import "std/print.rin"

// Every value lives somewhere, and where it lives decides how long it lasts.
```

```
// There are three answers in this language and you pick one every time you
// declare something. This lesson is about telling them apart by looking.

// Static storage. One copy, created before main runs, alive until the program
// exits. `static` here means "not visible outside this file", the same as C.
g_counter: static i32 = 0;

// Also static storage, but visible to other translation units.
g_shared: i32 = 100;

bump: proc() -> i32 = {
    // Automatic storage. Created when `bump` is entered, gone when it returns.
    // A fresh one every call -- which is why this always prints 1.
    local: i32 = 0;
    local += 1;

    // The static lives across calls, because it was never destroyed.
    g_counter += 1;

    printfmt("  local = {}   g_counter = {}\n", local, g_counter);
    return local;
}

// This is the mistake the lesson exists to name.
leak_a_local: proc() -> *i32 = {
    doomed: i32 = 42;
    // `doomed` stops existing the moment this returns. The address stays a
    // number, and the number stays valid-looking, and reading through it is
    // undefined behaviour. Nothing here will tell you.
    return doomed&;
}

main: proc() -> i32 = {
    printfmt("static lives at   {}\n", cast(g_shared&, uintptr));

    on_the_stack: i32 = 1;
    printfmt("automatic lives at {}\n", cast(on_the_stack&, uintptr));

    printfmt("calling bump three times:\n");
    bump();
    bump();
    bump();

    // The two addresses above are far apart, and that distance is the point:
    // statics live in the executable's own data, the stack is somewhere else
    // entirely, and the operating system moved it when the program started.
```

```
    return 0;
}

// What to notice
//
// 1. `local` printed 1 every time and `g_counter` counted up. Same syntax,
//    same type, entirely different lifetime -- decided by where it was
//    declared, not by how it was written.
//
// 2. The static address and the stack address are nowhere near each other.
//    They come from different regions of the process. Run it again and the
//    static address barely moves while the stack one jumps around.
//
// 3. `leak_a_local` compiles. It is never called here, because calling it and
//    reading the result is a real bug -- the kind that works on your machine
//    for a year. There is no garbage collector to save you and no borrow
//    checker to stop you. Knowing which storage you are in *is* the skill.
//
//    Build it and the C compiler does say something:
//
//      warning: address of stack memory associated with local variable
//               'doomed' returned
//
//    Worth knowing why that happened. rin hands its output to a C compiler,
//    so you get that compiler's analysis for free -- and this is one of the
//    few lifetime mistakes it can see, because the return is right there. Hide
//    the same address in a struct field and store it somewhere, and nothing
//    warns. Do not learn "the compiler will tell me".
//
// The third answer -- memory you ask for and manage yourself -- is the next
// lesson. It exists because automatic storage dies too soon and static storage
// has to be sized before the program runs, and real programs need neither.
//
// Try this
//
// - call `leak_a_local` and print the result through the pointer. It will
//   probably print 42. That is the worst possible outcome, because it teaches
//   you the wrong lesson; call another procedure first and print it again.
// - move `local` out of `bump` to file scope and watch the output change
// - add a second static to `bump` and confirm both persist independently
```

11 — Arenas and containers

> Ask once for a lot of memory, hand out pieces, throw the lot away.

The idea

An arena holds a block of memory and a cursor. Allocating moves the cursor forward and hands back what it passed. There is no free list, no per-allocation header, and no way to release one piece on its own — you release everything at once by moving the cursor back.

That sounds like a limitation and is mostly a relief. Most memory in a real program has the same lifetime as some phase: a frame, a level load, a parse. When the phase ends the whole arena resets, and nothing can be leaked or freed twice because nothing was individually owned.

std's containers are all built on it, and all take the arena as their first argument:

container	what it is
`array<T>`	a fixed length, set when it is reserved
`vec<T>`	grows on append, doubling its storage
`map<T>`	string keys to values
`list<T>`, `stack<T>`, `queue<T>`	linked nodes, pushed from the arena

What the lesson shows

- one arena feeding an `array`, a `vec` and a `map`
- `squares.data[0..squares.length]`: a container's storage as a checked slice
- `vec<f32>get` returning an `option` instead of reading past the end
- `memops_arena_clear` freeing every container at once

Why this is here

Every container in std takes an arena. Once you see why, the signature stops looking like boilerplate and starts looking like the allocation decision being made where it belongs: at the call site, by you.

Before you move on

You can say what happens to an `array`, a `vec` and a `map` when their arena is cleared, and why none of them has a destructor.

Source: `src/lessons/11_arenas_and_array.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/11_arenas_and_array.rin — 67 lines

```
import "std/print.rin"
import "std/array.rin"
import "std/vec.rin"
import "std/map.rin"
import "std/memops.rin"

// Every std container takes the arena it allocates from. The call site
// decides where the memory comes from, and one reset frees it all.

main: proc() -> i32 = {
    arena: memops_arena = {};
    memops_arena_initialize(arena&);

    // array<T>: a length fixed when it is made. `data` is the raw storage;
    // `data[0..length]` turns it into a checked slice.
    squares: array<i32> = array<i32>reserve(arena&, 5);
    view: []i32 = squares.data[0..squares.length];
    for (i: u64 = 0; i < view.length; i += 1) {
        view[i] = cast(i * i, i32);
    }
    printfmt("squares: {} {} {}\n", view[0], view[2], view[4]);

    // vec<T>: grows as you append, doubling its storage from the arena.
    path: vec<f32> = {};
    for (step: i32 = 1; step <= 6; step += 1) {
        vec<f32>append(arena&, path&, step * 0.5);
    }
    printfmt("path has {} points, last is {}\n", path.length, path.data[path.length - 1]);

    // A lookup that can miss returns an Option rather than garbage.
    seventh: option<f32> = vec<f32>get(path&, 7);
    printfmt("path[7] exists: {}\n", seventh.has_value);

    // map<T>: string keys to values.
    ages: *map<i32> = map<i32>create(arena&);
    map<i32>set(arena&, ages, "ada", 36);
    map<i32>set(arena&, ages, "linus", 54);
    linus: *i32 = map<i32>get(arena&, ages, "linus");
    printfmt("{} entries; linus is {}\n", map<i32>length(arena&, ages), linus*);
```

```
    // Nothing above is freed one by one: clearing the arena resets its cursor,
    // and every container that lived in it is gone at once.
    memops_arena_clear(arena&);
    return 0;
}

// What to notice
//
// 1. No container here owns memory the way a C++ vector does. Each one holds a
//    pointer into the arena, and the arena decides when it all goes away.
//
// 2. `squares.data` is a raw `^i32`: the struct knows the length, the
//    pointer does not. `data[0..length]` puts them back together as a slice,
//    and from then on `view[i]` is checked.
//
// 3. `vec<f32>get` returns an Option. Index 7 is past the end, so `has_value`
//    is 0 -- nothing was read. `path.data[7]` would have read whatever was there.
//
// 4. `step * 0.5` is an i32 times a float literal: the int becomes an f32,
//    the one implicit conversion lesson 01 allows between kinds.
//
// Try this
//
// - write `view[5] = 1;` with a variable 5 and run it; then the same through
//   `squares.data[5]`, which nothing checks
// - append a hundred points and print `path.border` as it grows
// - look up a key that is not in the map: `map<i32>get` returns null
```

12 — Generics

> One definition, many types, resolved at compile time.

The idea

`Box: struct<T>` is a template with a hole in it. `Box<i32>` fills the hole and the compiler emits a real struct holding an `i32`. `Box<f32>` emits a different one. Neither costs anything at runtime — there is no boxing, no vtable and no type tag, because the decision was made while compiling.

The naming is worth noting: a proc belonging to a generic type is written `Box<T>make`, with the type parameter attached to the type, not the proc. That is why it reads as "Box's make" rather than a free function that happens to mention Box.

What the lesson shows

- a generic struct with one type parameter
- `Box<T>make` and `Box<T>unwrap`, both generic procs
- instantiating at two different types in the same program
- the generated C has one concrete struct per instantiation

Why this is here

`array<T>`, `vec<T>`, `option<T>`, `map<K,V>` and `slice<T>` are all this. So is every container you will write. And because rin has no methods, generics plus proc pointers are the whole of its abstraction budget.

Before you move on

You can write a generic type with two parameters and instantiate it.

Source: `src/lessons/12_generics_box.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/12_generics_box.rin — 23 lines

```
import "std/print.rin"

Box: struct<T> = {
    value: T;
}
```

```
Box<T>make: proc<T>(value: T) -> Box<T> = {
    box: Box<T> = {};
    box.value = value;
    return box;
}

Box<T>unwrap: proc<T>(box: Box<T>) -> T = {
    return box.value;
}

main: proc() -> i32 = {
    a: Box<i32> = Box<i32>make(12);
    b: Box<f32> = Box<f32>make(3.5);

    printfmt("{} {}\n", Box<i32>unwrap(a), Box<f32>unwrap(b));
    return 0;
}
```

13 — Printing, and extending it

> `printfmt` finds a `print` overload per type, including yours.

The idea

`print<T>` is one overload per type. std provides them for the scalars and its own containers; you add one for your type by declaring `print: proc<YourType>` and `printfmt` finds it exactly the way it finds the built-in ones.

No registration, no interface to implement, no runtime lookup. The overload is selected while compiling, from the type of the argument.

What the lesson shows

- `printfmt` with `{}` holes rather than `%d` / `%s` codes
- a user-written `print` overload for an enum
- the overload being picked automatically at the call site

An enum needs no overload to be printed. Without one, `{}` prints the member's name. The overload here is only needed for a different spelling, `red` instead of `Red`.

Why this is here

This is the pattern for "the language knows how to handle my type" in a language with no methods and no interfaces. It generalises: overload per type, resolved at compile time, extended by declaring one more.

Before you move on

You can add a `print` overload for a struct of your own.

Source: `src/lessons/13_std_print.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/13_std_print.rin — 42 lines

```
import "std/print.rin"

Color: enum = {
    Red,
    Green,
```

```
}

// `print<T>` is one overload per type, and you can add your own. printfmt finds
// this the same way it finds the built-in ones. Without it, an enum prints its
// member name ("Green"); this one replaces that with its own spelling.
print: proc<Color>(c: Color) -> void = {
    if (c == Color.Red) {
        print_cstr("red");
    } else {
        print_cstr("green");
    }
}

main: proc() -> i32 = {
    count: i32 = 42;
    ratio: f32 = 0.5;
    name: ^const char = "std print";

    // The long way: one call per piece, picking the overload by hand.
    print<^const char>("std print says ");
    print<i32>(count);
    print<^const char>("\n");

    // The short way. Each `{}` takes the next argument and prints it with the
    // `print<T>` for that argument's type -- so there is no format letter to get
    // wrong, and no cast to f64 the way a C printf would need for a f32.
    printfmt("{} says {} and {}\n", name, count, ratio);

    // Including your own types, without printfmt knowing anything about them.
    printfmt("color is {}\n", Color.Green);

    // Both of these are resolved while compiling. `{}` never exists at run time:
    // the compiler splits the string, works out each argument's type, and emits
    // a single printf where it can. Look in build/gen to see what came out.
    printfmt("{} + {} = {}\n", count, count, count + count);
    return 0;
}
```

14 — Enum-sized tables

> An array whose length is an enum's member count, checked by the compiler.

The idea

`Stat<>` is the enum's reflection record and `.count` is how many members it declares. The compiler knows that number as soon as the enum is parsed, so it can size an array with it: `[Stat<>.count]i32`.

The alternative is a hand-written `STAT_COUNT` constant sitting next to the enum, which is correct exactly until someone adds a member and forgets. Here there is nothing to forget, because there is nothing to keep in step.

What the lesson shows

- `Stat<>.count` used as an array length
- indexing that array by casting the enum member
- adding a member grows the table automatically

Why this is here

This is the first place reflection earns its keep, and it does so at compile time with no runtime cost at all. The array is exactly as fast as one with a literal length, because after compilation it is one.

Before you move on

You can add a fourth `Stat` member and predict what changes.

Source: `src/lessons/14_enum_sized_tables.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/14_enum_sized_tables.rin — 21 lines

```
import "std/print.rin"

Stat: enum = {
    Health,
    Stamina,
    Focus,
}
```

```
main: proc() -> i32 = {
    // `Stat<>` is the enum's reflection record, and `.count` is how many members
    // it declares. The compiler knows that number once the enum is parsed, so it
    // can size an array with it -- there is no hand-written STAT_COUNT to drift
    // out of step when a member is added.
    values: [Stat<>.count]i32 = {};
    values[Stat.Health] = 100;
    values[Stat.Stamina] = 50;
    values[Stat.Focus] = 25;

    printfmt("stats={} count={}\n", values[Stat.Health] + values[Stat.Focus], Stat<>.count);
    return 0;
}
```

15 — The C boundary

> `cinclude` sends a header to C; `external` declares a name to rin.

The idea

These are two separate things and confusing them is the most common early mistake.

`cinclude "stdio.h"` arranges for that header to reach the C compiler. It brings **no name into rin**. That is why `printf` still has to be declared with `proc[external]` even though `<stdio.h>` is right there.

`external` means "C already owns this definition". rin type-checks your calls against the declaration you wrote and emits nothing for it — no prototype, no body. If your declaration disagrees with the real header, that is your bug to find, and the C compiler will usually find it.

What the lesson shows

- `cinclude` and `proc[external]` used together, and why both are needed
- `#define` passing straight through to the C preprocessor
- a varargs C function declared with `...`
- `^const char` as the C string type

Every pointer that crosses into C is a `^T`. A header cannot say whether `char *` means one character or a million, so rin does not guess: the pointer stays raw and unchecked, and your code turns it into a `*T` (`p[0]&`) or a slice (`p[0..n]`) once it knows which it is. A string literal is a `^const char` for the same reason.

Why this is here

Every real rin program has a boundary like this — njinn calls D3D11, cgltf and miniaudio through exactly these declarations. The discipline of writing them out is what lets the compiler check the calls at all.

Before you move on

You can declare and call a C function that is not in this lesson.

Source: `src/lessons/15_c_surface.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/15_c_surface.rin — 17 lines

```
cinclude "stdio.h"
#define I_LEARN_BONUS 7

// `external` means C already owns the declaration. The rin compiler can type-check
// calls without emitting another definition for it.
//
// Every C function needs one of these before you can call it. `cinclude` sends
// the header to the C compiler; it does not bring any name into rin. That is why
// `printf` is declared here too, even though <stdio.h> is included above.
puts: proc[external](text: ^const char) -> i32 = {}
printf: proc[external](fmt: ^const char, ...) -> i32 = {}

main: proc() -> i32 = {
    puts("this call is checked by rin and provided by C");
    printf("bonus=%d\n", I_LEARN_BONUS);
    return 0;
}
```

16 — Reflection: the record

> `T<>` is a record the compiler emits describing the type.

The idea

For any type, `T<>` gives a value with its name, size, alignment, kind and member count. It is not a runtime lookup and not a hashtable — it is a `const` struct the compiler wrote into the output, and reading it is reading a global.

If nothing in your program mentions `T<>`, nothing is emitted. Reflection costs what you use.

What the lesson shows

- `.name`, `.size`, `.align`, `.count` on a struct
- the same four on an enum
- `.kind` telling the two apart
- the record being ordinary data you can pass around

Why this is here

Everything in the next three lessons is built from this record. It is worth seeing the plain fields before the ones that lead to arrays of other records.

Before you move on

You can print the size of a type two ways and get the same number.

Source: `src/lessons/16_reflection_metadata.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/16_reflection_metadata.rin — 33 lines

```
import "std/print.rin"
import "std/reflect.rin"

Payload: struct = {
    id: i32;
    weight: f32;
    label: *const char;
}
```

```
Mode: enum = {
    Idle,
    Busy,
}

main: proc() -> i32 = {
    // `Type<>` hands back one `reflect` record whatever the type is: struct,
    // union or enum. `kind` is what tells them apart, and `count` means fields
    // for a struct and members for an enum.
    printfmt("type={} kind={} fields={} size={}\n",
        Payload<>.name, reflect_kind_name(Payload<>.kind), Payload<>.count, Payload<>.size);
    printfmt("type={} kind={} members={}\n",
        Mode<>.name, reflect_kind_name(Mode<>.kind), Mode<>.count);

    // The payload that only one kind has lives behind `variant`: a struct's
    // fields, an enum's values. Asking for the wrong arm is a compile error
    // wherever the compiler can see which kind you have -- try changing this to
    // `Payload<>.variant.values` and rebuilding.
    printfmt("field0={}:{}\n", Payload<>.variant.fields[0].name, Payload<>.variant.fields[0].t
    printfmt("field2={} ptr_depth={}\n",
        Payload<>.variant.fields[2].name, Payload<>.variant.fields[2].pointer_depth);
    printfmt("mode0={}={}\n", Mode<>.variant.values[0].name, Mode<>.variant.values[0].value);
    return 0;
}
```

17 — Reflection: fields

> Walking a struct's members, and the attributes you can attach to them.

The idea

`T<>.variant.fields` is an array of one record per field: name, type name, offset, size, and the attribute string you wrote after `@`.

That last part is the interesting one. `id: i32 @ "save,editor"` attaches text to a field that the compiler carries into the reflection table, where your own code can read it. Serialisation, editor interfaces and diffing are all "walk the fields, look at the tags, do something" — and none of it needs the compiler to know what "save" means.

What the lesson shows

- iterating `fields[i]` up to `.count`
- `.offset` matching what lesson 07 taught you to predict
- attributes read back as a string you can search
- a nested struct field, and reaching its own record through `.info`

Why this is here

This is the mechanism njinn uses for its editor and its save format. It is also the strongest argument for reflection existing at all: the alternative is a hand-maintained table that drifts.

Before you move on

You can find a field by name and print its offset.

Source: `src/lessons/17_reflection_fields.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/17_reflection_fields.rin — 41 lines

```
import "std/print.rin"
import "std/reflect.rin"

Inner: struct = {
    x: f32;
    y: f32;
```

```
}

Thing: struct = {
    id: i32 @ "save,editor";
    name: *const char;
    scores: [3]i32;
    origin: Inner;
}

main: proc() -> i32 = {
    printfmt("{} has {} reflected fields\n", Thing<>.name, Thing<>.count);
    for (i: u64 = 0; i < Thing<>.count; i += 1) {
        field: *const reflect_field = Thing<>.variant.fields[i]&;
        printfmt("  {} : {} size={} attrs={}\n",
            field*.name,
            field*.type,
            field*.size,
            field*.attrs);
    }

    // `type` is only a name. `info` is the link: the record for the field's own
    // type, so a walk can descend instead of stopping at a string. It is null
    // for anything without a table of its own -- a builtin, a pointer to C.
    printfmt("nested:\n");
    for (i: u64 = 0; i < Thing<>.count; i += 1) {
        field: *const reflect_field = Thing<>.variant.fields[i]&;
        if (field*.info != null) {
            printfmt("  {} -> {} ({} fields)\n",
                field*.name, field*.info*.name, field*.info*.count);
        } else {
            printfmt("  {} -> (builtin, no record)\n", field*.name);
        }
    }
    return 0;
}
```

18 — One record, three kinds

> Structs, unions and enums are all described by the same shape.

The idea

There is one record type. `.kind` says which sort of type it describes, and `.variant` is a union whose live arm follows from that: `fields` for a struct or union, `values` for an enum.

Reading the wrong arm is the mistake this design invites, so the compiler helps: completion only offers the arm that is live for the kind you have.

What the lesson shows

- a struct, a union and an enum described side by side
- `.kind` switched on to pick the right arm
- `variant.values` for the enum, with each member's name and number
- a union whose fields all sit at offset 0

Why this is here

The union at offset 0 is a nice check on lesson 07: reflection tells you the same thing the layout rules would, and now you can read it at runtime instead of working it out.

Before you move on

You can write one proc that prints any type's members, whatever its kind.

Source: `src/lessons/18_reflection_one_record.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/18_reflection_one_record.rin — 68 lines

```
import "std/print.rin"
import "std/reflect.rin"

// Three different kinds of type, described by one record apiece.
Point: struct = {
    x: f32;
    y: f32;
```

```
}

Word: union = {
    as_int: i32;
    as_float: f32;
}

Slot: enum = {
    Empty = -1,
    Filled = 0,
}

// A walker does not need to know what it was handed. `kind` tells it, and the
// checked accessors hand back the payload only when it matches -- so this one
// procedure describes all three types without a cast in sight.
describe: proc(meta: *const reflect) -> void = {
    fields: ^const reflect_field = reflect_fields(meta);
    values: ^const reflect_value = reflect_values(meta);

    printfmt("{}: kind={} members={} size={}\n",
        meta*.name, reflect_kind_name(meta*.kind), meta*.count, meta*.size);

    if (fields != null) {
        for (i: u64 = 0; i < meta*.count; i += 1) {
            printfmt("    {}: {} at offset {}\n", fields[i].name, fields[i].type, fields[i].of
        }
    }
    if (values != null) {
        for (i: u64 = 0; i < meta*.count; i += 1) {
            printfmt("    {} = {}\n", values[i].name, values[i].value);
        }
    }
}

main: proc() -> i32 = {
    describe(Point<>&);
    describe(Word<>&);
    describe(Slot<>&);

    // A union's members all start at offset zero -- they share storage. That is
    // why a union is its own kind rather than a struct with a flag: a walker
    // that only knew structs would lay these out as if they were adjacent and
    // write nonsense.
    printfmt("union overlap: {} and {} both at offset {}\n",
        Word<>.variant.fields[0].name,
        Word<>.variant.fields[1].name,
```

```
        Word<>.variant.fields[1].offset);

    // Asking for the wrong arm is caught at compile time wherever the kind is
    // known. Uncomment this and rebuild to see the diagnostic:
    //
    //     printfmt("{}\n", Point<>.variant.values[0].name);
    //
    // Where the kind is only known at run time -- a `*const reflect` that was
    // passed in, as in describe() above -- reflect_fields and reflect_values are
    // the guard: they return null rather than reinterpreting the pointer.
    if (reflect_values(Point<>&) == null) {
        printfmt("asking a struct for enum values yields null, not garbage\n");
    }
    return 0;
}
```

19 — Where this leads

> Generics and reflection together: code shaped by types.

The idea

The last lesson puts the two halves next to each other. A generic type gives you one definition instantiated per type; reflection gives you a description of each instantiation. Together they are enough to write code that adapts to a type it was not written for — a serialiser, a comparison, an editor field — with the adaptation resolved while compiling.

That is the whole of rin's metaprogramming. There is no macro system, no compile-time execution, and no `comptime` block. Generics plus reflection plus the preprocessor is the budget, deliberately.

What the lesson shows

- a generic type with an occupancy flag, instantiated twice
- reflection over the instantiated type
- the generated C, with one concrete struct per instantiation

Why this is here

You now have every tool the language offers. What remains is not more features — it is judgement about which of these to reach for, which the book chapters and your own programs will teach better than a lesson can.

Before you move on

You can describe what you would build with these two together.

Source: `src/lessons/19_metaprogramming_shape.rin`. The file itself carries "What to notice" and "Try this" sections — read the code first, then come back here.

The code

src/lessons/19_metaprogramming_shape.rin — 40 lines

```
import "std/print.rin"

Slot: struct<T> = {
    value: T;
    occupied: b32;
}
```

```
Slot<T>some: proc<T>(value: T) -> Slot<T> = {
    slot: Slot<T> = {};
    slot.value = value;
    slot.occupied = 1;
    return slot;
}

Slot<T>none: proc<T>() -> Slot<T> = {
    return {};
}

main: proc() -> i32 = {
    health: Slot<i32> = Slot<i32>some(10);
    empty: Slot<f32> = Slot<f32>none();

    // Each instantiation gets its own record under its monomorphised name, so
    // Slot<i32> and Slot<f32> reflect separately rather than sharing one entry
    // for the template. `Type<>` and the generated `Type_reflect` global are
    // the same thing; both are checked the same way.
    printfmt("{} {} {} {}\n",
        Slot_i32_reflect.name,
        Slot_i32_reflect.count,
        Slot_f32_reflect.name,
        Slot_f32_reflect.count);

    // An exit code says whether the program succeeded, not what it computed.
    // Print the result and return 0; a shell reading `10` here would think
    // something went wrong.
    if (health.occupied && !empty.occupied) {
        printfmt("health holds {}, empty holds nothing\n", health.value);
    }
    return 0;
}
```